Bozza di progetto – Footsies RL

Oct 28, 2026

Obiettivo e domanda di ricerca

Il progetto addestra agenti per Footsies con un solo algoritmo, PPO in self-play, e studia come la funzione di reward cambia ciò che l'agente impara, misurandone forza e robustezza, non solo il win rate.

La domanda guida è: **un agente che vince contro avversari semplici è anche difficile da sfruttare?** Il paper di FootsiesGym mostra che le due cose possono divergere (un metodo può battere gli altri in scontro diretto ed essere comunque il più sfruttabile). Il progetto riproduce in piccolo questa analisi, cambiando la funzione di reward con lo stesso algoritmo, e la collega alla teoria del corso (modellazione, bias e varianza).

Il confronto principale è tra quattro funzioni di reward con lo stesso PPO, scritto da noi in PyTorch (senza RLlib):

1. **Reward terminale**: ±w alla vittoria o sconfitta, 0 al timeout (baseline).
2. **Reward denso sulla guardia con budget**: bonus a ogni calo della guardia dell'avversario, sottratto dal premio finale (già nella libreria).
3. **Shaping basato su potenziale**: da scrivere come wrapper; in teoria non cambia la policy ottima.
4. **Vittoria scalata sul tempo**: vincere prima vale di più, resta a somma zero (wrapper).

Il pool di checkpoint (PFSP), l'entropia decrescente e le altre ablation sono estensioni se resta budget (vedi Esperimenti e ablation).

Perché Footsies e formulazione come POMDP

Footsies è un gioco a due giocatori, a somma zero e a informazione imperfetta, quindi si presta a essere formalizzato come POMDP e come problema multi-agente competitivo (cap. 2.4 e 11 del libro).

- **Stato**: configurazione completa del gioco per entrambi i giocatori, incluse le feature che un giocatore non vede dell'altro.
- **Osservazione**: vettore di 85 dimensioni (88 con le azioni di carica) con la parte comune, le feature di sé, quelle dell'avversario e un blocco "privilegiato" solo per sé (dash disponibile, progresso della carica, azione precedente).
- **Azioni**: 6 discrete di default (NONE, BACK, FORWARD, ATTACK, BACK+ATTACK, FORWARD+ATTACK), 9 con le azioni di carica per le mosse speciali.

- **Reward**: a somma zero, ±10 alla vittoria o sconfitta, 0 al timeout (4000 step); opzionale reward denso per i guard break (vedi la sezione sulla funzione di reward).
- **Parziale osservabilità**: l'agente non vede le intenzioni dell'avversario, che deve prevedere. Con `action_delay` alto il gioco diventa quasi pura previsione.

Come nel progetto Briscola preso a modello, la scelta di formulazione va scritta esplicitamente nella relazione e nelle slide: stato, osservazione, azioni, transizioni, reward, e la catena s→o→a.

Dinamica, inizializzazione e terminazione

Il simulatore Unity applica le regole del gioco; per quanto risulta dal paper non ci sono elementi casuali nelle transizioni (da verificare), quindi la casualità di un episodio viene dalle policy dei due giocatori. Ogni passo dell'ambiente sono 4 frame di gioco (`frame_skip`) e l'azione scelta viene eseguita 8 frame dopo (`action_delay`): l'ambiente tiene una coda di azioni in attesa e inserisce la propria azione precedente nell'osservazione privilegiata.

Inizializzazione. A ogni `reset()` l'ambiente chiama `ResetGame` sul server Unity; lato Python non c'è nessuna casualità e l'argomento `seed` viene ignorato. Si azzerano la coda delle azioni (riempita di NONE), la carica speciale, il budget del reward (riportato a w) e lo stato dell'encoder. La posizione iniziale dei giocatori è decisa dal server e il paper non la descrive: va verificata nella repo del gioco, ma probabilmente è sempre la stessa. Conseguenza: ogni partita parte dalla stessa configurazione, quindi la varietà dei dati dipende solo dalla stocasticità delle policy (per questo l'entropia conta) e la valutazione alterna i lati P1 e P2.

Terminazione. Un episodio termina quando un giocatore è sconfitto (`is_dead` ; secondo il paper, quando è colpito da un attacco speciale) oppure è troncato a 4000 passi, circa 267 secondi di gioco a 60 Hz: in quel caso entrambi ricevono 0. Le due cose vanno trattate in modo diverso nel PPO: alla sconfitta il valore dello stato successivo è zero, alla troncatura si fa bootstrap con il valore stimato dal critico (l'ambiente vettorizzato sostituisce l'ultima osservazione con quella di reset, quindi come approssimazione si usa V dell'osservazione precedente), altrimenti si introduce un bias perché lo stato non è davvero terminale (vedi Bias e varianza).

Funzione di reward: perché, limiti e conseguenze

Cosa è definito. Il reward è a somma zero: +w al vincitore e -w allo sconfitto (w = `win_reward_scaling_coeff`), 0 a entrambi al timeout. Opzionalmente `guard_break_reward` x: a ogni calo della guardia dell'avversario (stando al codice, a ogni attacco bloccato) l'attaccante riceve +x e il difensore -x, con il vincolo 3x < w; con

`use_reward_budget` lo shaping viene sottratto dal premio finale, così il totale di una partita vinta non supera w. Attenzione: nel paper il default è w = 10, ma nella libreria installata `win_reward_scaling_coeff` vale 1,0, quindi va impostato esplicitamente a 10 (oggi `train_ppo.py` non lo fa).

Perché questa scelta. Il premio terminale coincide con l'obiettivo vero (vincere il round) e non cambia il problema: la forma del reward non introduce bias. La somma zero rende il gioco puramente competitivo (cap. 11) e il self-play ben definito: ciò che guadagna un agente lo perde l'altro. Il timeout a 0 tratta il pareggio come neutro. La reward terminale, che è il default dell'ambiente e quello del paper, è la baseline del confronto.

Conseguenze sull'apprendimento. Il segnale è sparso e ritardato: arriva solo a fine episodio, fino a 4000 passi dopo; con gamma 0,995 un'azione compiuta più di circa 920 passi prima della fine pesa meno dell'1%, quindi il credit assignment è difficile e la stima del ritorno ha varianza alta. Con il timeout a 0, contro un avversario passivo (il No-op) ogni partita non vinta è un pareggio senza segnale, e il paper osserva che le policy migliori diventano passive o reattive. La scala conta: il coefficiente di entropia (0,025) e quello della value loss (1,0) sono tarati su w = 10, con w = 1 il bonus di entropia peserebbe relativamente 10 volte di più. Il reward denso accelera l'apprendimento ma cambia l'obiettivo: l'agente potrebbe imparare a farsi bloccare gli attacchi per far scendere la guardia invece di vincere. È un bias introdotto volontariamente in cambio di meno varianza; l'ablation su `guard_break_reward` lo misura (variante 2 del confronto).

Varianti confrontate e idee escluse.

- Shaping basato su potenziale (variante 3): il potenziale è la differenza di guardia tra i due giocatori, da definire dopo aver verificato come evolve la guardia (se si rigenera va tenuto conto). Il bonus è F = gamma · Phi(s′) - Phi(s), con potenziale opposto per i due giocatori, quindi resta a somma zero. In teoria non cambia la policy ottima, a differenza del reward denso (Ng et al., 1999; non è nel libro).
- Vittoria scalata sul tempo (variante 4): il premio è ±w moltiplicato per un fattore decrescente da 1 a 0,5 sull'orizzonte (valori da fissare), così vincere prima è meglio; resta a somma zero. Rischio: l'agente preferisce partite rapide e rischiose.
- Penalità sul timeout (facoltativa): scoraggia la passività ma, se è uguale per entrambi i giocatori, rompe la somma zero e fa perdere il significato esatto di best response ed exploitability. Si prova solo come caso a parte, dichiarandolo.

Ambiente e setup tecnico

L'ambiente è già installato nella repo e funziona con agenti casuali (`test.py` , `visual_test.py`); c'è una prima baseline PPO su RLlib (src/train_ppo.py); manca il resto del codice di training e valutazione.

- **Ambiente**: pacchetto `footsies-gym` (API PettingZoo), simulatore Unity headless via gRPC, vettorizzabile con `num_envs`. Il paper riporta circa 19.000 step/s con un solo processo e circa 52.500 con 4 processi e 128 ambienti su una workstation da 24 core.
- **Configurazione di base**: `frame_skip` 4, `action_delay` 8, `max_t` 4000, reward di vittoria 10 (valore del paper; il default della libreria installata è 1,0, quindi va impostato `win_reward_scaling_coeff` a 10). Le variazioni (delay, carica speciale, reward denso) sono leve per gli esperimenti.
- **Dove si lavora**: sviluppo e test brevi sul Mac (`platform="mac"`), run lunghe sul cluster Orfeo (SLURM, partizione THIN, senza GPU perché la rete è piccola) (`platform="linux"`). Windows non è supportato.
- **Librerie**: PyTorch. PPO, wrapper di reward, valutazione e avversari sono scritti da noi; RLlib resta solo come baseline di riferimento (train_ppo.py) per validare la nostra implementazione.
- **Versione di Python**: fissata a 3.12 con `.python-version`, la stessa su Mac e su Orfeo. PyTorch e la libreria dell'ambiente funzionano con questa versione (verificato su entrambi).

Avversari fissi (scala di difficoltà)

Servono quattro avversari, dal più debole al più forte, per dare un'unità di misura comune a tutte le valutazioni; il progetto Briscola ne usava cinque. Tre sono banali da scrivere (Random, No-op e Back sono un campionamento uniforme e due azioni costanti) e uno è già nella libreria: il bot a regole FootsiesBot.

Avversario	Logica	Cosa misura
Random	Azione uniforme tra le 6 disponibili (da scrivere)	Soglia minima di competenza
No-op	Non fa mai nulla (azione costante)	Se l'agente sa iniziare lo scontro (nel paper i policy migliori diventano passivi)
Back	Indietreggia sempre (azione costante)	Se l'agente sa inseguire e colpire un avversario che fugge
FootsiesBot	IA a regole del gioco originale (BattleAI), riscritta nella libreria	Livello contro un benchmark usato anche dal paper

FootsiesBot (footsiesgym/footsies/game/footsies_bot.py) usa lo stato della partita e un po' di casualità, quindi è il più vicino a come gioca un umano. Non è una rete: prende lo stato della partita, quindi serve un adattatore, già scritto in src/opponents.py

(FootsiesBotOpponent). Funziona solo con return_fight_state_in_infos=True e num_envs=1, perché con gli ambienti vettorizzati le info sono vuote: si usa quindi nella valutazione, un ambiente alla volta, e non nel training vettorizzato. Copre solo le 6 azioni base.

Algoritmo: PPO scritto da noi in self-play

Gli agenti condividono osservazione, azioni, rete e iperparametri di PPO e differiscono per la funzione di reward; l'avversario è sempre la stessa rete in self-play.

Implementazione: PPO scritto da noi (PyTorch). Rete MLP 2×256 con ReLU e due teste (policy e valore), raccolta su ambienti vettorizzati della libreria (`num_envs`), vantaggio con GAE, loss con clipping più termine di valore e bonus di entropia, 8 epoche su minibatch. Nel self-play semplice la stessa rete gioca entrambi i lati e usa i dati di entrambi i giocatori; nel pool si usano solo i dati del lato che sta imparando. La baseline RLlib già presente (`train_ppo.py`) resta come riferimento: prima di fidarsi del nostro codice si controlla che le curve contro il Random siano simili. Si parte dall'esempio CleanRL ufficiale di FootsiesGym (adattato da ppo_pettingzoo_ma_atari): ha già l'ambiente vettorizzato con slot p1 e p2 che condividono la rete, GAE, loss e le metriche di diagnostica (varianza esplicata, KL, frazione di clipping). Si cambiano iperparametri (learning rate 1e-4, rollout 64, entropia come da paper), trattamento della troncatura, reward e valutazione. Collegamento al libro: vantaggio e actor-critic (cap. 9.3.2-9.3.3); GAE e clipping non sono nel libro.

1. PPO con self-play semplice. Iperparametri di partenza dal paper: rete MLP 2×256 con ReLU, gamma 0,995, GAE lambda 0,95, clip 0,3, 8 epoche con 8 minibatch, rollout di 64 passi, 48 ambienti in parallelo, learning rate 1e-4 che decresce a 0, ottimizzatore Adam, coefficiente della value loss 1,0, norma massima del gradiente 0,5 (Tabella 3 del paper). Il coefficiente di entropia è fisso (0,025) in tutte le run sul confronto delle reward; la variante decrescente (da 0,1 a 0) è un'ablation secondaria.

2. Estensione facoltativa: PPO con pool di checkpoint. Ogni partita è giocata contro la policy corrente con probabilità 0,8 e contro uno snapshot del pool altrimenti. Gli snapshot (fino a 16, uno ogni circa 614.000 step) sono scelti con peso proporzionale a (1 - p_i)^(1/2), dove p_i è la frequenza di vittoria stimata contro lo snapshot i: si gioca di più contro chi ci batte. Con il PPO scritto da noi la scelta dell'avversario è un campionamento fatto a ogni reset dell'ambiente, senza toccare learner o moduli. L'esempio di RLlib (MixManagerCallback) aggiunge avversari quando il win rate supera una soglia: è un curriculum e non un PFSP, quindi resta solo come spunto.

Scala degli esperimenti. Il paper usa 25 milioni di step per 5 seed. Per il progetto si usa un solo seed per run, scelta dichiarata nella sezione Valutazione. Questo libera budget: il numero di step per run si fissa dopo aver misurato la velocità reale di addestramento (vedi

Budget di calcolo), anche fino ai 25 milioni del paper se ci sta. Se serve semplificare, si riducono prima ablation e best response.

Bias e varianza nell'algoritmo

In RL bias e varianza hanno un significato diverso dall'apprendimento supervisionato: non riguardano la complessità del modello, ma la qualità dello stimatore del target che guida l'aggiornamento (il ritorno, il vantaggio, il gradiente della policy), calcolato da poche traiettorie campionate. Si può usare il reward realmente osservato (in media corretto, ma molto rumoroso: poco bias, molta varianza) oppure una stima appresa del valore futuro (stabile, ma sbagliata quanto sbaglia il critico: poca varianza, ma bias). È lo spettro tra Monte Carlo e TD del cap. 6 (n-step); in questo progetto il compromesso si regola con lambda, gamma e il clipping.

```
A_t = \sum_{k=0}^{\infty} (\gamma \lambda)^k \, \delta_{t+k}, \qquad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)
```

Scelta	Effetto	Bias	Varianza	Libro
Ritorno Monte Carlo (REINFORCE)	usa tutto il ritorno G_t	nessuno	alta: dipende da tutti i reward fino a 4000 passi	9.3
Baseline V(s)	sottrae una stima del valore dello stato	nessuno (aspettativa nulla)	più bassa	9.3.1
Bootstrapping con il critico	sostituisce il futuro con V(s′)	sì, pari all'errore del critico	più bassa	8.3, 9.3.3
GAE con lambda 0,95	media pesata di TD a più passi	intermedio: lambda 0 è il TD a un passo, lambda 1 è Monte Carlo	intermedia	6 (spettro n-step); GAE non è nel libro
Sconto gamma 0,995	orizzonte efficace di circa 200 passi	sì: si ottimizza il ritorno scontato, non il vincere	più bassa	2.5-2.7

Scelta	Effetto	Bias	Varianza	Libro
Clipping e 8 epoche (epsilon 0,3)	riusa gli stessi dati più volte	sì: si ottimizza una surrogata, non il gradiente esatto	più bassa per campione; il rapporto di importanza è la fonte di varianza off-policy	5 (off-policy); PPO non è nel libro
Bonus di entropia	spinge verso policy più casuali	sì, voluto; sparisce alla fine nella variante decrescente (0,1 a 0)	non la riduce direttamente	
Troncatura a 4000 passi	se trattata come fine vera	sì; si evita con il bootstrap su V		

Come si legge per PPO. PPO parte dall'actor-critic (cap. 9.3.3), che abbassa la varianza di REINFORCE a prezzo del bias del critico, e con GAE regola il compromesso con un solo parametro. Il problema è tipico di questo gioco: con reward solo finale ed episodi lunghi, un Monte Carlo puro avrebbe varianza altissima (per questo REINFORCE è stato scartato), mentre un TD a un passo propaga il reward di una vittoria un passo alla volta e si affida a un critico che deve stimare un valore molto lontano.

Perché non un metodo basato sul massimo (Q-learning, DQN): PPO non soffre della maximization bias del cap. 5.7, che nasce dal prendere il massimo di stime rumorose, e produce una policy stocastica, necessaria in un gioco a strategie miste.

Come si misura. Con due ablation (lambda 0 e 1 contro 0,95; gamma 0,99 contro 0,995) e con la diagnostica durante il training: varianza esplicata del critico (quanto del ritorno spiega V, indice del suo bias), deviazione standard del vantaggio per batch (indice di varianza), KL e frazione di campioni tagliati dal clipping. Anche le varianti di reward si leggono così: il reward denso e lo shaping danno un segnale più frequente (meno varianza del ritorno) a costo di un possibile bias nell'obiettivo. Le ipotesi da verificare, non risultati: lambda 1 più lento e rumoroso (varianza), lambda 0 più stabile ma più facilmente bloccato su policy peggiori (bias). Lo stesso vocabolario vale per la valutazione: la best response approssimata dà un bias per difetto sulla sfruttabilità, e un solo seed non permette di stimare la varianza tra addestramenti.

Valutazione e metriche

Ogni agente è valutato su tre livelli (contro avversari fissi, in scontro diretto e contro una best response), perché il solo win rate contro euristiche nasconde la sfruttabilità.

Livello	Come	Cosa dice
Avversari fissi	1000 partite per avversario, lati P1 e P2 alternati, win/draw/loss con intervallo di confidenza al 95%	Livello di abilità sulla scala di difficoltà
Scontro diretto	Matrice tra le varianti di reward, un modello per variante, molte partite con i lati alternati	Quale variante batte quale
Exploitability approssimata	Si congela l'agente finale e si addestra contro di esso una best response con PPO (1 per agente; 3 solo se il budget lo permette)	Limite inferiore di quanto l'agente è sfruttabile
Curve di apprendimento	Win rate contro gli avversari fissi ogni N step	Velocità e stabilità dell'addestramento

Quattro scelte di metodo da dichiarare:

- **Evaluation stocastica, non greedy.** In un gioco con strategie miste una policy deterministica è sfruttabile per costruzione, quindi si valuta campionando dalla distribuzione. Il progetto Briscola usava la scelta greedy; qui non è adatto.
- **Un solo seed per run.** Il paper ne usa 5, noi uno solo: le differenze tra metodi non sono testabili statisticamente e restano descrittive. Gli intervalli di confidenza sulle partite di valutazione misurano il rumore della valutazione, non la variabilità tra addestramenti diversi. Per mostrare che un risultato non è un caso si riportano le curve su più checkpoint della stessa run e, se resta budget, si ripete con un secondo seed il confronto più importante.
- **Best response come limite inferiore.** La best response è approssimata, quindi stima per difetto la sfruttabilità reale; va detto nella relazione.
- **Metrica comune.** Il ritorno non si può confrontare tra reward diverse: tutte le varianti si giudicano con win rate contro gli avversari fissi, scontro diretto e best response, misurati con il reward terminale (anche la best response si addestra con quello).

Esperimenti e ablation

Il nucleo sono due confronti, le funzioni di reward e il compromesso bias-varianza; le ablation sull'ambiente e le estensioni si aggiungono in ordine di priorità se resta tempo di calcolo.

Priorità	Esperimento	Cosa varia	Domanda
Core	Reward	terminale; denso sulla guardia con budget; shaping a potenziale; vittoria scalata sul tempo	Quale reward produce l'agente più forte e meno sfruttabile?
Core	Bias e varianza	GAE lambda 0 e 1 contro 0,95; gamma 0,99 contro 0,995	Come cambiano velocità e stabilità con più bias (TD) o più varianza (Monte Carlo)?
Secondaria	Entropia	fissa (0,025) contro decrescente (0,1 a 0)	La regolarizzazione aiuta la robustezza?
Secondaria	Azione di carica	`use_special_charge_action` on/off	L'agente scopre e usa le mosse speciali?
Secondaria	Ritardo d'azione	`action_delay` 0, 8, 12	Con delay 0 il gioco diventa reattivo e fragile, come nel paper?
Facoltativa	Pool di checkpoint	self-play contro snapshot passati con campionamento prioritario	Il pool riduce la sfruttabilità?
Facoltativa	Penalità sul timeout	penalità simmetrica, fuori dalla somma zero	Scoraggia la passività, e cosa perde la valutazione?

Tutte le run usano un solo seed, quindi i risultati sono descrittivi. Tutte le run usano la stessa rete di base, così che le differenze si attribuiscano alla variabile cambiata.

Budget di calcolo

Con un solo seed le run sono 15 (23 con 3 best response per agente), ma il tempo di ciascuna dipende dalla velocità reale di addestramento: il primo passo è misurarla e poi decidere quanti step per run ci stanno. Unità di costo: S = step di una run PPO.

Gruppo di run	N. run	Costo
Confronto reward: 4 funzioni di reward con PPO, 1 seed ciascuna	4	4 S
Best response: 1 per agente finale (3 per agente)	4 (12)	2 S (6 S)
Bias e varianza: lambda 0 e 1, gamma 0,99	3	3 S
Ablation secondarie: entropia decrescente, carica speciale, delay 0 e 12	4	4 S
Totale	15 (23)	13 S (17 S)

La best response è contata come 0,5 S per run: è un'assunzione da verificare. Le estensioni facoltative (pool, penalità sul timeout) non sono nel totale e costano circa 1 S ciascuna più la best response. Con S = 5 milioni di step il totale è 65 milioni (85 milioni con 3 best response per agente); con S = 25 milioni, come nel paper, è 325 milioni (425 milioni).

Le velocità del paper (circa 19.000 step/s con un processo, circa 52.500 con 4 processi) misurano solo il simulatore, non l'addestramento della rete. Misurate sul nostro PPO completo (48 partite, 3.072 step per update): circa 6.700 step/s sul Mac e circa 4.050 su un nodo THIN di Orfeo con 4 thread. Quattro job contemporanei su THIN fanno 3.800-4.500 step/s ciascuno, senza rallentamenti. Il tempo si divide tra aggiornamento della rete (56% sul Mac, 63% su Orfeo) e simulatore (37% e 25%). Una run da 25 milioni di step dura quindi circa 1 ora sul Mac e 1,7 ore su THIN (2,2 su GENOA). GENOA fa circa 3.100 step/s, stabili su 3 job, ed è la seconda scelta; EPYC è la più lenta (circa 750 step/s, misurata con una versione precedente del codice) e non si usa. Il piano gira su Orfeo con un job per run, in parallelo: il tempo reale è circa quello di una run più la coda, non la somma. Se non ci sta, si taglia in quest'ordine: ablation facoltative, best response a 1 seed, step per run.

Diagnostica e analisi del comportamento

Le percentuali di vittoria non bastano a capire cosa ha imparato l'agente, quindi si registrano e si analizzano le partite, come nella parte di diagnostica del progetto Briscola.

- **Log delle partite**: per ogni passo si salvano osservazione, azione scelta, probabilità delle azioni ed esito, così da riguardare singole decisioni.
- **Distribuzione delle azioni** nel tempo: quanto attacca, blocca, avanza, indietreggia.
- **Uso delle mosse speciali**: numero medio per partita, che nel paper è quasi zero senza le azioni di carica.
- **Iniziativa contro il no-op**: l'agente ingaggia l'avversario fermo o resta passivo? Il paper segnala che i policy migliori diventano sempre più reattivi.

- **Video di alcune partite** con la build grafica (`visual_test.py` mostra come aprirla), da usare nelle slide.
- **Diagnostica di bias e varianza**: varianza esplicata del critico, deviazione standard del vantaggio per batch, KL e frazione di campioni tagliati dal clipping, registrate a ogni aggiornamento.

Questa parte è quella che più aiuta all'orale: permette di dire cosa fa l'agente e perché, non solo quanto vince.

Collegamento con il libro del corso

Ogni scelta del progetto si appoggia a un capitolo delle dispense, da citare esplicitamente in relazione e all'orale.

Elemento del progetto	Capitolo del libro
Stato, osservazione, belief, parziale osservabilità	2.4 POMDP
Reward terminale, ritorno, fattore di sconto	2.5-2.7 Goals, returns, episodi
Politiche e funzioni di valore	2.8-2.9
Approssimazione con rete neurale	8 Approximate solution methods
Baseline e riduzione della varianza (base teorica di PPO)	9.3 e 9.3.1 REINFORCE e baseline
Vantaggio e actor-critic (alla base di PPO)	9.3.2-9.3.3
Teorema del policy gradient	9.2 e 10.1
Ottimalità globale del policy gradient	10.2
Self-play, non stazionarietà, best response, exploitability	11 Multi-agent RL e teoria dei giochi
Bias e varianza: Monte Carlo, TD, n-step, GAE, bootstrapping	6 n-step bootstrapping; 8.3 semi-gradient; 9.3, 9.3.1, 9.3.3
Perché non Q-learning o DQN (maximization bias)	5.7 Maximization bias e double learning

PFSP e la best response non sono nel libro (il cap. 11 ne dà solo l'idea generale). PPO è scritto da noi come evoluzione dell'actor-critic (cap. 9.3.2-9.3.3): aggiornamenti limitati dal clipping, motivati dall'alta varianza del policy gradient puro (REINFORCE, cap. 9.3). GAE e

clipping non sono nel libro e vanno spiegati a parte; PFSP e la best response sono applicazioni delle idee del cap. 11.

Struttura della repo e consegne

Le consegne sono il codice con un README riproducibile e la presentazione per l'orale, entrambe costruite sulla repo già esistente.

```
FootsiesRL/
  README.md            come installare, lanciare train e valutazione
  pyproject.toml, uv.lock   dipendenze gestite con uv (uv sync, uv run)
  configs/             un file per esperimento (reward, seed, delay,
entropia, lambda, gamma)
  src/
    ppo.py             PPO scritto da noi (rete, GAE, loss, aggiornamento)
    train.py           ciclo di addestramento, self-play, pool e checkpoint
    train_ppo.py       baseline su RLlib, solo come riferimento
    rewards.py         varianti di reward (wrapper: potenziale, vittoria
scalata sul tempo)
    pool.py            (facoltativo) campionamento prioritario degli snapshot
(PFSP)
    opponents.py       Random, No-op, Back e adattatore per FootsiesBot
    evaluate.py        partite, intervalli di confidenza, matrice head-to-
head
    best_response.py   addestramento della best response
    diagnostics.py     log delle partite e statistiche sulle azioni
  results/             tabelle e figure
  test.py  visual_test.py   già presenti
```

La presentazione segue la struttura che ha funzionato nel progetto Briscola:

1. Introduzione e motivazione (perché Footsies, perché è un POMDP).
2. Formulazione: stato, osservazione, azioni, inizializzazione, reward (scelte, limiti, miglioramenti).
3. Avversari fissi e agenti appresi.
4. Addestramento: PPO scritto da noi in self-play, bias e varianza dell'algoritmo.
5. Confronto tra le reward, valutazione e diagnostica.
6. Risultati e limitazioni.

Calendario, rischi e limiti

Il lavoro procede in sei fasi; le date dipendono dalla scadenza dell'esame, che non conosco, e si fissano appena la indichi.

1. Setup e verifica dell'ambiente (versione di Python), misura della velocità reale di addestramento sul server, avversari fissi e valutazione di base.
2. Implementazione di PPO e validazione contro la baseline RLlib, poi self-play semplice e prime curve di apprendimento.
3. Varianti di reward (potenziale, vittoria scalata sul tempo) e confronto con la reward terminale.
4. Best response e matrice degli scontri diretti.
5. Bias e varianza (lambda, gamma), ablation secondarie e diagnostica del comportamento.
6. Slide, README e prova dell'orale.

Rischio	Come si affronta
Il PPO scritto da noi ha un bug o impara peggio della baseline RLlib	Si confrontano le curve contro il Random con train_ppo.py e si controllano loss, KL e frazione di campioni tagliati dal clipping
Tempo di calcolo insufficiente	Meno step e meno ablation; ambienti vettorizzati e, se serve, più processi
Le mosse speciali non vengono scoperte	Si attivano le azioni di carica come ablation
Le varianti di reward non producono differenze visibili	Si riporta come risultato e se ne discutono le cause (bias e varianza, un solo seed); se resta budget si ripete il confronto con un secondo seed
L'agente diventa passivo	Si misura contro il no-op e lo si discute come limite
Exploitability rumorosa	Più seed per le best response e intervalli di confidenza

Limiti dichiarati fin d'ora: con un solo seed per run i risultati non separano l'effetto del metodo dalla variabilità tra addestramenti, quindi sono descrittivi; la best response approssima per difetto la sfruttabilità; la policy usa solo l'osservazione corrente (senza rete ricorrente né belief esplicito), quindi la parziale osservabilità è trattata in modo limitato; Footsies isola la sola fase neutrale, molto più semplice di un gioco di combattimento commerciale.a libe